

8. IMAGE SEGMENTATION

AIM:

To study and implement the image segmentation techniques.

INTRODUCTION:

Segmentation subdivides an image into its constituent regions or objects. The level of detail to which the subdivision is carried depends on the problem being solved. Fundamental problem in image segmentation is to partition an image into regions. Segmentation algorithm for monochrome images generally are based on one of two basic categories dealing with properties of intensity values.

1.Discontinuity

2. Similarity

EDGES:

Derivative operator is used to identify the edges. First order derivative can be used to detect the presence of an edge. The sign of the second order derivative can be used to determine whether an edge pixel lie on the dark or light side of an edge. Second order Derivatives are sensitive to noise. Sobel operators introduce some smoothing and give more importance to the center point.

POINT DETECTION:

Point detection is based on the second order derivative. It uses Laplacian mask.

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
I=cv2.imread(r"C:\Users\DELL\Pictures\IPLAB_Final\test14.tif")
T=np.mean(I)
k=np.array([[ -1, -1, -1], [-1,8, -1], [-1, -1, -1]])
out=cv2.filter2D(I, -1,k)
real=np.abs(out)
```

```

T=250
segmented = np.where(real >= T, 255, 0).astype(np.uint8)

plt.figure(figsize=(10,20))
plt.subplot(1,3,1)
plt.title("input")
plt.imshow(I, cmap='gray')
plt.axis("off")
plt.subplot(1,3,2)
plt.title("mask")
plt.imshow(real, cmap='gray')
plt.axis("off")
plt.subplot(1,3,3)
plt.title("Segmented ")
plt.imshow(segmented, cmap='gray')
plt.axis("off")

plt.show()

```

output:

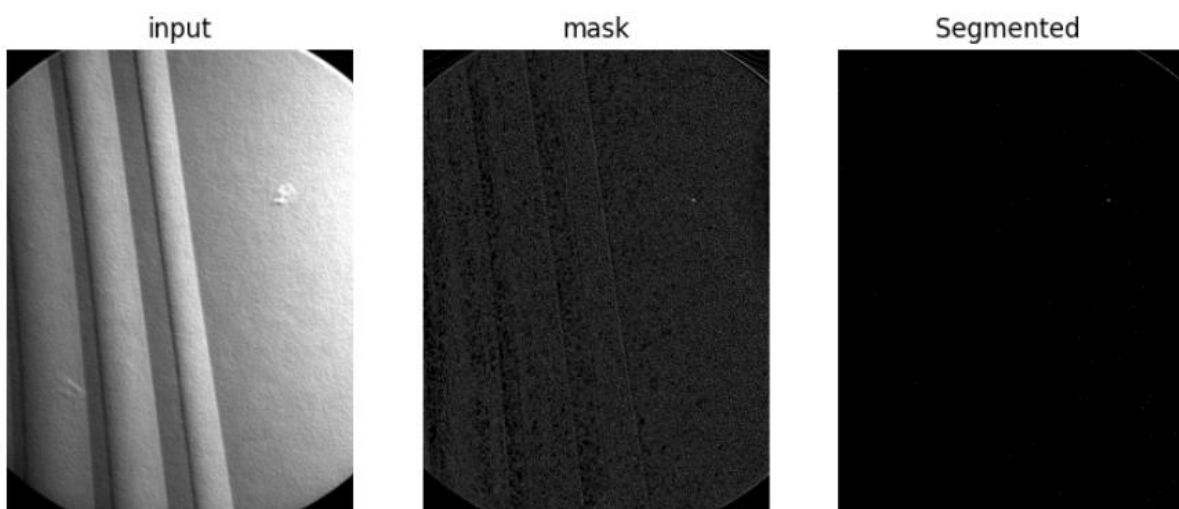


Figure:(a)Input image for point detection (b) The input image is enhanced using a Laplacian mask to highlight edges (c) thresholding on the filtered result produces the segmented image showing only strong edge features.

LINE DETECTION:

It is based on the mask operation. The preferred direction is weighted by with a larger Coefficient. The masks can be defined for vertical, horizontal and diagonal direction lines .

Code:

```
##horizontal
import cv2
import numpy as np
import matplotlib.pyplot as plt

a=cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH09\Fig0905(a)(wirebond-
mask).tif",cv2.IMREAD_GRAYSCALE)

h=np.array([[ -1, -1, -1],[2,2,2],[ -1, -1, -1]])
b=cv2.filter2D(src=a,ddepth=-1,kernel=h)
b_abs=np.abs(b)
t=230
o=(b_abs>t).astype(np.uint8)*255
plt.figure(figsize=(10,20))
plt.subplot(1,2,1)
plt.imshow(a,cmap='gray')
plt.subplot(1,2,2)
plt.imshow(b,cmap='gray')
```

output:

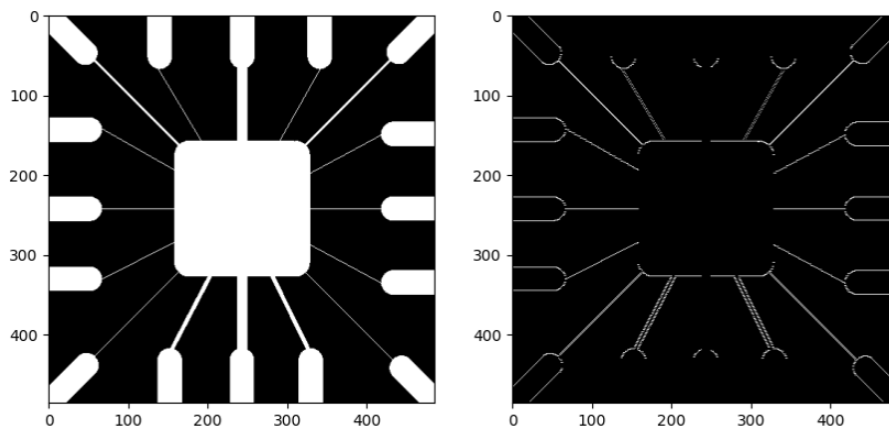


Figure: (a) shows the original wire-bond mask, while image (b) highlights the horizontal edges of the structure using a horizontal edge detection filter.

Code:

```
##-45 deg
import cv2
import numpy as np
import matplotlib.pyplot as plt

a=cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH09\Fig0905(a)(wirebond-
mask).tif",cv2.IMREAD_GRAYSCALE)

h=np.array([[2,-1,-1],[-1,2,-1],[-1,-1,2]])
b=cv2.filter2D(src=a,ddepth=-1,kernel=h)
b_abs=np.abs(b)
t=240
o=(b_abs>t).astype(np.uint8)*255
plt.figure(figsize=(10,20))
plt.subplot(1,2,1)
plt.imshow(a,cmap='gray')
plt.subplot(1,2,2)
plt.imshow(b,cmap='gray')
```

output:

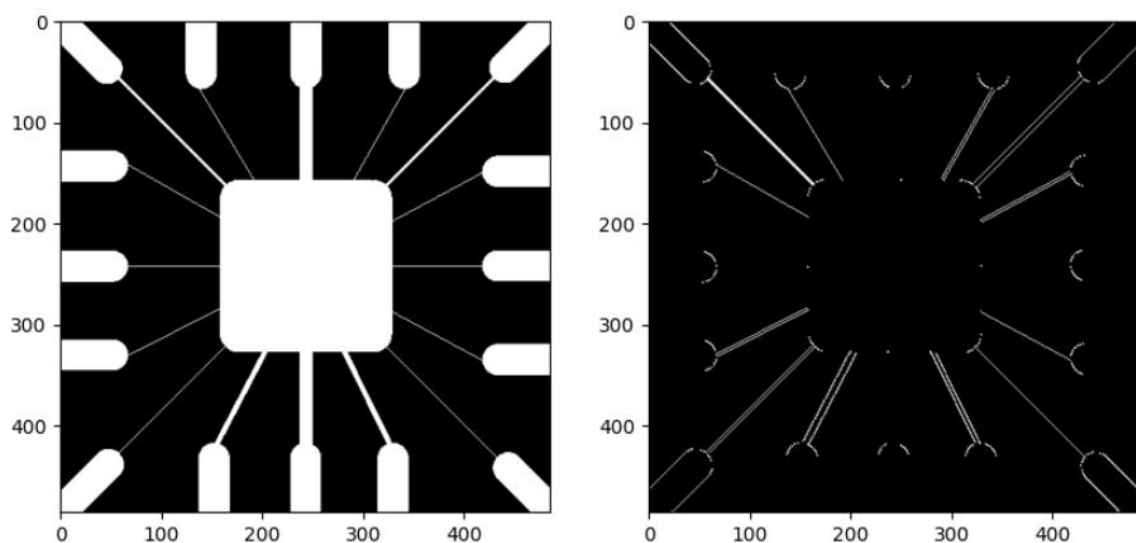


Figure: (a) shows the original wire-bond mask, while image (b) highlights the -45 edges of the structure using a horizontal edge detection filter.

Code:

```
##+45deg
import cv2
import numpy as np
import matplotlib.pyplot as plt

a=cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH09\Fig0905(a)(wirebond-
mask).tif",cv2.IMREAD_GRAYSCALE)

h=np.array([[ -1, -1,2],[ -1,2, -1],[2, -1, -1]])

b=cv2.filter2D(src=a,ddepth=-1,kernel=h)

b_abs=np.abs(b)

t=240

o=(b_abs>t).astype(np.uint8)*255

plt.figure(figsize=(10,20))
plt.subplot(1,2,1)
plt.imshow(a,cmap='gray')
plt.subplot(1,2,2)
plt.imshow(b,cmap='gray')
```

output:

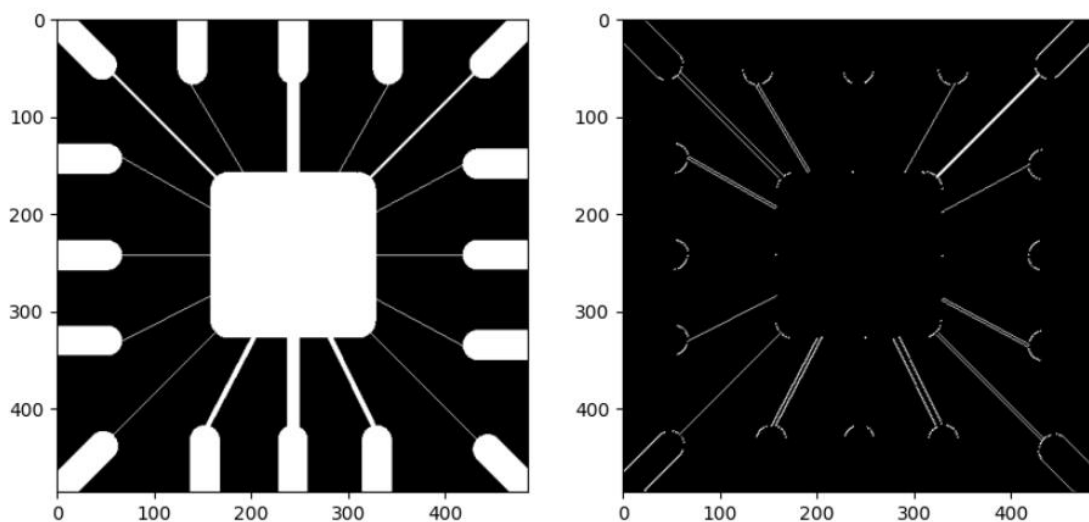


Figure: (a) shows the original wire-bond mask, while image (b) highlights the +45 edges of the structure using a horizontal edge detection filter.

Code:

```
##vertical
import cv2
import numpy as np
import matplotlib.pyplot as plt

a=cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH09\Fig0905(a)(wirebond-
mask).tif",cv2.IMREAD_GRAYSCALE)

h=np.array([[ -1,2, -1],[ -1,2, -1],[ -1,2, -1]])
b=cv2.filter2D(src=a,ddepth=-1,kernel=h)
b_abs=np.abs(b)
t=240
o=(b_abs>t).astype(np.uint8)*255
plt.figure(figsize=(10,20))
plt.subplot(1,2,1)
plt.imshow(a,cmap='gray')
plt.subplot(1,2,2)
plt.imshow(b,cmap='gray')
```

output:

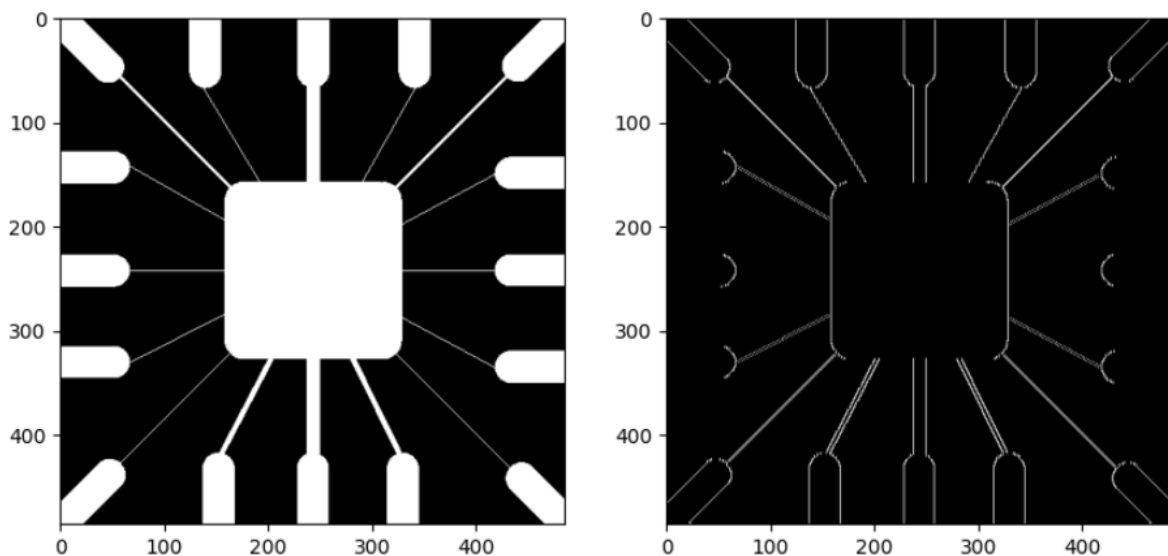


Figure: (a) shows the original wire-bond mask, while image (b) highlights the vertical edges of the structure using a horizontal edge detection filter.

THRESHOLDING:

Consider that the above histogram corresponds to the image $f(x, y)$, composed of the light objects on the dark background. such a way that object and background intensity values are grouped into two dominant modes. The way to extract the object from the background is to select the threshold, T that separates these modes. Then, any point (x, y) in the image at which $f(x, y) \geq T$ is called an object point. Otherwise the point is called a background point.

Otsu Method or Optimal threshold method:

Otsu's thresholding method involves iterating through all the possible threshold values and calculating a measure of spread for the pixel levels each side of the threshold, i.e. the pixels that either fall in foreground or background. The aim is to find the threshold value where the sum of foreground and background spreads is at its minimum. The matlab uses the function "graythresh" for optimal thresholding.

Code: Global thresholding.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

a=cv2.imread(r"C:\Users\DELL\Pictures\IP
folder\IPLAB_2025\DIP3E_Original_Images_CH09\Fig0940(a)(rice_image_with_in
tensity_gradient).tif")

t=np.mean(a)

dt=1

while True:
    g1=a[a>t]
    g2=a[a<=t]
    m1=np.mean(g1) if g1.size > 0 else 0
    m2=np.mean(g2) if g2.size > 0 else 0
    newt=0.5*(m1+m2)
    if abs(newt-t)<dt:
        break
    t=newt
```

```
o =(a>=t).astype(np.uint8) * 255

plt.figure(figsize=(10,20))
plt.subplot(1,2,1)
plt.imshow(a,cmap='gray'),plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(o,cmap='gray'),plt.axis('off')
```

Output:

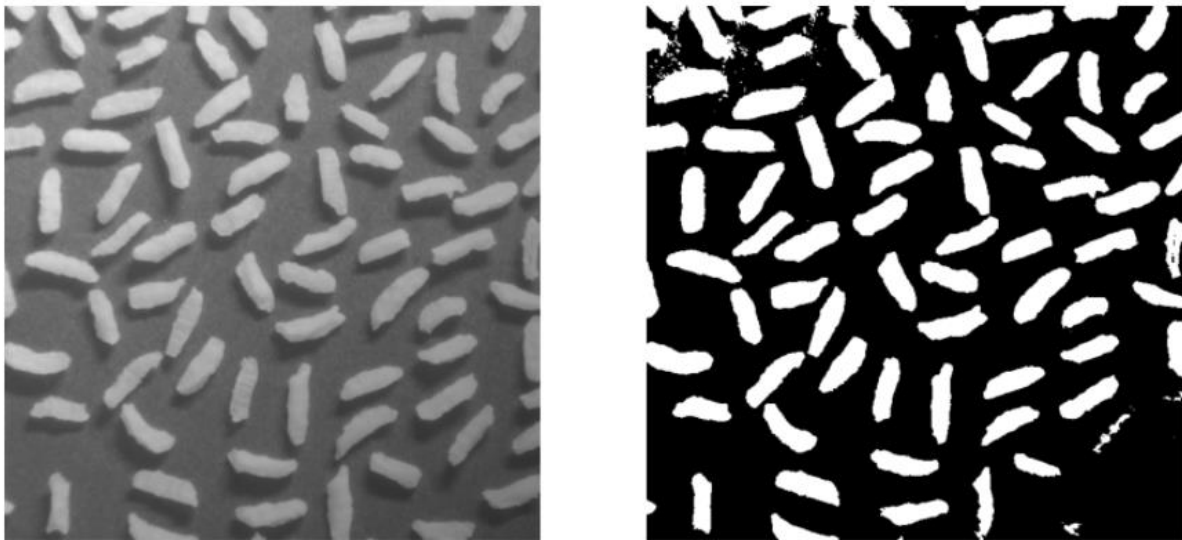


Figure:(a) Input image for global thresholding.(b)Using global thresholding with iterative selection, the rice grains are separated from the background, but due to the intensity gradient some regions are not segmented perfectly.